Othello-Spieler `spieler.wendo`

Kurzbeschreibung der Lösung

Gruppe `wendo` – *Name, Matrikelnummer*

1 Überblick

Der Spieler implementiert die vorgegebene Schnittstelle `OthelloSpieler`. Die Zugberechnung erfolgt durch eine **Alpha-Beta-Suche in der NegaScout-Variante** über einen Spielbaum, kombiniert mit **iterativer Tiefensuche unter Zeitkontrolle**. Die Stellungsbewertung ist eine handgewichtete Heuristik, die je nach Spielphase unterschiedliche Kriterien betont. Sämtliche Klassen liegen im Unterpaket `spieler.wendo`; die Spielerklasse heißt `Spieler`.

2 Architektur

Der Code ist nach dem Prinzip der Trennung der Zuständigkeiten (Single Responsibility) in fünf Klassen gegliedert. Nur `Spieler` kennt die Framework-Typen; `Board`, `MiniMax`, `Evaluate` und `TimeManager` sind vollständig framework-unabhängig und dadurch isoliert testbar.

- `Spieler` – Adapter zur Wettkampf-Umgebung: übersetzt `Farbe` und `Zug` in die interne Darstellung und zurück, pflegt das eigene Spielbrett und behandelt Sonderfälle (Passen, ungültiger Gegnerzug → `ZugException`).
- `Board` – Spielregeln und Spielstand (Zuggenerierung, Ausführen inkl. Umdrehen, Endstellungserkennung).
- `MiniMax` – Suchverfahren (NegaScout, Zugsortierung, iterative Tiefensuche, Zeitabbruch).
- `Evaluate` – Stellungsbewertung.
- `TimeManager` – Aufteilung der Gesamtbedenkzeit auf die einzelnen Züge.

3 Brettdarstellung

Das 8×8-Brett wird als flaches `int`-Array der Länge 64 gespeichert (Index = Zeile $\cdot\, 8 +$ Spalte; Werte LEER/SCHWARZ/WEISS). Diese Darstellung ist speichereffizient und lässt sich billig kopieren, was für die Suche entscheidend ist: Die Suche arbeitet nach dem Prinzip *copy-make*, d. h. für jeden Zug wird eine unabhängige Kopie erzeugt, auf der der Zug ausgeführt wird – die Ausgangsstellung bleibt unverändert. Das vermeidet fehleranfällige „RückgängigLogik.

4 Zugberechnung

Bei jedem Aufruf von `berechneZug` wird zunächst der letzte Gegnerzug auf dem eigenen Brett nachgespielt (Synchronisation), anschließend das Zeitbudget für diesen Zug bestimmt und die Suche gestartet.

Suchverfahren (NegaScout). Die Suche ist als *Negamax* formuliert: Beide Spieler maximieren aus ihrer eigenen Sicht, und bei jedem Halbzug wird das Vorzeichen der Kindbewertung negiert. Auf dieser Basis verwendet der Spieler **NegaScout** (Prinzipalvariantensuche): Der erste – durch die Zugsortierung vielversprechendste – Zug wird mit vollem Suchfenster untersucht, alle weiteren nur mit einem *Null-Fenster* als billige „Ist dieser Zug überhaupt besser?Probe. Nur wenn diese Probe fehlschlägt, wird der Zug voll nachgesucht. Dadurch werden bei guter Zugsortierung deutlich weniger Knoten expandiert als bei einfacher Alpha-Beta-Suche.

```
negascout(brett, tiefe, alpha, beta, seite):
  if tiefe == 0 or spielEnde: return bewerte(brett, seite)
```

```
zuege = sortiere(legaleZuege(seite))
best = -inf; erster = true
for zug in zuege:
  kind = brett.kopie(); kind.fuehreAus(zug, seite)
  if erster: score = -negascout(kind, tiefe-1, -beta, -alpha, gegner)
  else:
    score = -negascout(kind, tiefe-1, -alpha-1, -alpha, gegner)   # Scout-Probe
    if alpha < score < beta:
      score = -negascout(kind, tiefe-1, -beta, -score, gegner)    # Nachsuche
  best = max(best, score); alpha = max(alpha, best)
  if alpha >= beta: merkeKiller(tiefe, zug); break                 # Beta-Schnitt
return best
```

Zugsortierung. Da Alpha-Beta bzw. NegaScout nur dann effizient beschneiden, wenn der beste Zug zuerst untersucht wird, werden die Züge an jedem Knoten heuristisch sortiert:

- **Statische Feldbewertung** – Sortierung nach einer festen Gewichtstabelle (Ecken sehr hoch, ecknahe „X"/„C"-Felder negativ).
- **Prinzipalvariante** – über die iterative Tiefensuche wird der beste Zug der vorherigen (flacheren) Suche in der nächsten Iteration zuerst probiert.
- **Killer-Heuristik** – pro Suchtiefe werden bis zu zwei Züge gemerkt, die zuletzt einen Beta-Schnitt ausgelöst haben, und an Geschwisterknoten bevorzugt probiert. Diese Maßnahme reduzierte die Knotenzahl messbar um ca. 37 %.

Iterative Tiefensuche und Zeitmanagement. Statt sofort mit fester Tiefe zu suchen, wird nacheinander mit Tiefe $1, 2, 3, \ldots$ gesucht. Das hat zwei Vorteile: Es liegt jederzeit ein vollständiges Ergebnis vor (wichtig für die Zeitkontrolle), und jede flachere Iteration verbessert die Zugsortierung der nächsten. Für jeden Zug berechnet `TimeManager` ein Budget aus der *verbleibenden* Bedenkzeit geteilt durch die *geschätzte Restzugzahl* (mit Sicherheitsreserve). Läuft das Budget während einer Iteration ab, wird die Suche über eine leichtgewichtige Ausnahme sofort abgebrochen; zurückgegeben wird der beste Zug der letzten *vollständig* abgeschlossenen Tiefe. Die Tiefe ist dabei nur eine Obergrenze – real begrenzt die Zeit; im Endspiel mit wenigen freien Feldern wird die Partie faktisch bis zum Ende ausgerechnet.

5 Stellungsbewertung

Die Bewertungsfunktion liefert einen Wert aus Sicht der ziehenden Seite und ist *antisymmetrisch* ($\text{bewerte}(b, \text{seite}) = -\text{bewerte}(b, \text{gegner})$), was für die Negamax-Formulierung notwendig ist. Sie ist **phasenabhängig**:

- **Mittelspiel:** gewichtete Summe aus *Eckenkontrolle*, *Mobilität* (Differenz der Zugmöglichkeiten, normiert) und *Positionswert* (statische Feldtabelle). Die reine Steinzahl wird bewusst ignoriert – ein früher Steinvorsprung ist in Othello meist nachteilig.
- **Endspiel** (ab ca. 50 Steinen): Übergang zur *Steindifferenz* mit starker Gewichtung der Ecken, da nun die endgültige Steinzahl über Sieg oder Niederlage entscheidet.

6 Qualitätssicherung

Die regel- und suchkritischen Klassen sind durch 48 JUnit-Tests abgesichert (u. a. Regeln, Antisymmetrie der Bewertung, Such-Invarianten). Leistungsverbessernde Änderungen wurden anhand exakter Knotenzahlen gemessen und nur übernommen, wenn sie nachweislich halfen, ohne die Korrektheit (identisches Suchergebnis) zu verändern.